Plataforma de dados de cobrança no GCP

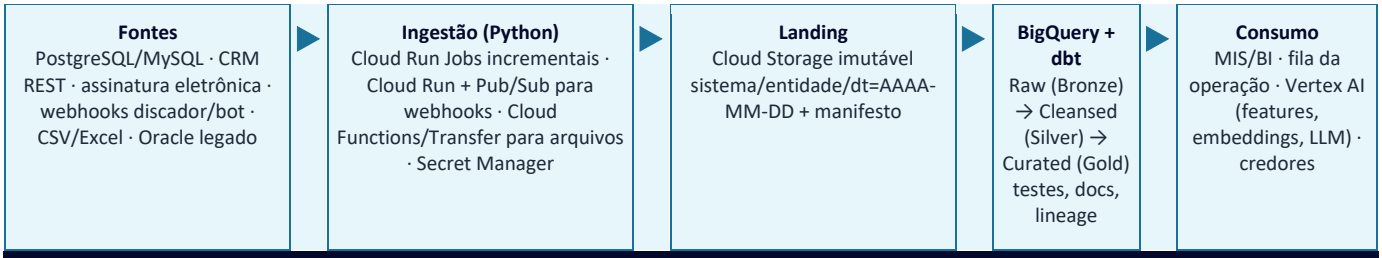
Como eu abordaria a plataforma de dados de uma operação de cobrança e recuperação de crédito

Proposta de arquitetura preparada para o processo seletivo de Arquiteto/Engenheiro de Dados da PeopleWare Tecnologia · Sidney Moraes · outubro de 2026 · projeto completo com código, testes e resultado de execução em **github.com/sysdevcns/plataforma-dados-cobranca-gcp**

1. Leitura do desafio

Uma operação de cobrança trabalha com carteiras de vários credores e fornecedores que mudam (discador, bot de WhatsApp, SMS, e-mail, plataforma de acordos, assinatura eletrônica). As fontes são heterogêneas: PostgreSQL/MySQL, CRM via API REST (tipo Pipedrive), assinatura eletrônica (tipo DocuSign), webhooks de discador e bot, planilhas CSV/Excel dos credores e Oracle legado. Os indicadores que importam (taxa de recuperação, aging, efetividade por canal) costumam não bater entre MIS, operação e credor; o status do CRM fica desatualizado; dado pessoal se espalha em planilhas e dashboards; o custo de nuvem cresce sem dono. A plataforma precisa garantir **um padrão único de ingestão** com linhagem, **regras de negócio em um só lugar e testadas**, **status derivado do fato** (pagamentos) e não do campo digitado, **dado pessoal em claro só onde é indispensável** e **custo visível desde o primeiro dia**.

2. Arquitetura proposta



Bloco	Decisão e justificativa
Landing e ingestão	Todo dado entra como arquivo imutável no Cloud Storage, com manifesto (origem, janela, contagem, hash). Extratores Python em Cloud Run Jobs: incremental por marca d'água, paginação, retry com backoff, segredo no Secret Manager. Webhooks entram por Cloud Run + Pub/Sub, desacoplando o pico do fornecedor da carga. Reprocessar é reler o arquivo, não refazer a extração.
BigQuery em três camadas com dbt	**Raw**: tabelas carregadas do landing + views de staging (renomeia, tipa, padroniza); dado pessoal em claro só aqui, em dataset com IAM restrito e policy tags. **Cleansed**: aging e dias de atraso, régua de cobrança como tabela de referência, status consolidado de parcela/acordo/contrato a partir dos pagamentos, efetividade de contato por catálogo de resultados. **Curated**: fct_parcelas, fct_contatos, dim_clientes (CPF e e-mail em hash com sal), dim_contratos, mart de recuperação diária e mart de efetividade por canal/fornecedor.
Orquestração	Cloud Scheduler → Cloud Run Jobs no início (um job por fonte, dbt build ao final); Cloud Composer (Airflow) quando houver dependências entre fontes, SLA por carteira e backfills. *dbt source freshness* como sensor.
CI/CD	GitHub Actions: *dbt parse* e testes de ingestão em todo PR sem credenciais; Slim CI em dataset efêmero via Workload Identity Federation; merge em main roda dbt build em produção; Terraform para datasets, buckets e IAM.
Dados para IA	Features de propensão a pagar e de melhor canal/horário a partir da Curated, publicadas no Vertex AI Feature Store com data de referência (sem vazamento de futuro). Transcrições e conversas de bot passam por Cloud DLP e viram embeddings (BigQuery VECTOR_SEARCH) para resumo, classificação de motivo e RAG com Gemini. O modelo prioriza a fila; a decisão continua humana (LGPD art. 20).
Governança e LGPD	Dado pessoal em claro só em Raw; Curated entrega hash com sal (Secret Manager), nome em iniciais, telefone só DDD, idade em faixa; teste automático falha o build se uma coluna proibida aparecer; opt-out materializado para régua e discador; lineage do dbt no Dataplex; acesso auditado; retenção alinhada ao contrato com o credor.
FinOps	Fatos particionados por data e clusterizados pelos filtros do BI; staging como view; maximum_bytes_billed por ambiente; cotas por projeto e usuário; lifecycle do bucket; billing export + INFORMATION_SCHEMA.JOBS para custo por consulta e por mart; reservas de slots só com histórico que prove economia.

3. Decisões e trade-offs

Decisão	Por quê	O que eu abro mão / quando reveria
dbt Core	Versionamento, testes como código, documentação e lineage, ambientes isolados, ecossistema. É o que permite dizer "o número do MIS tem teste".	Dataform se o time for 100% BigQuery sem necessidade dos pacotes do dbt; procedures só para rotinas pontuais.
Não copiar o Bronze no BigQuery	O landing físico é o Cloud Storage. Na Raw entra a tabela carregada + views de staging. Duplicar a tabela crua só dobra armazenamento e cria cópia que ninguém testa.	Em lake Parquet (Spark) o Bronze seria a ingestão física; aqui esse papel é do bucket.
Quando particionar	Fatos e marts diários por data quando passam de ~1 GB ou têm filtro de data dominante; abaixo disso clustering basta; dimensões só clustering; granularidade mensal em safra de vencimento evita partições minúsculas.	Particionar tabela pequena é anti-padrão. Reavaliar com bytes lidos por consulta.

Incremental sem MERGE	Webhooks chegam atrasados e com correção: reescrever as últimas N partições (insert_overwrite + copy_partitions) é idempotente e barato.	MERGE quando a fonte envia só deltas por chave e a tabela é grande demais para reescrever.
Status derivado do fato	O CRM informa o que alguém digitou; os pagamentos mostram o que aconteceu. A divergência vira indicador de qualidade da origem.	Exige acordo com a operação sobre tolerâncias; a regra vive em um modelo e muda por PR.
Oracle legado incremental com janela do DBA	Extrator Python (python-oracledb) por coluna de auditoria, em janela acordada (ex.: 01h-05h), lotes fixos, retomada por chave, limite de sessões; carga inicial em fatias por período.	GoldenGate/Datastream resolve CDC, mas custa licença e mexe no banco: só depois de medir volume e latência necessários. Sem coluna de auditoria: extração completa com hash por linha.
Scheduler antes de Composer	Composer tem custo fixo 24x7 e curva de operação; com poucas fontes independentes, Scheduler + Cloud Run Jobs resolve.	Migrar quando houver DAG real; os jobs já são contêineres, muda só o disparador.

4. Plano de 90 dias

Bloco	Entregas	Como se mede
1. Fundação e primeira carteira (sem. 1-4)	Projetos dev/prod, buckets, datasets, IAM via Terraform; padrão de ingestão com as duas fontes críticas (sistema de cobrança e discador); dbt com Raw/Cleansed/Curated para uma carteira; fct_parcelas, dim_contratos, mart de recuperação; CI e build diário.	Recuperação e aging da carteira piloto batendo com o MIS atual (ou divergência explicada); build diário sem intervenção por 2 semanas.
2. Todas as fontes e governança (sem. 5-8)	CRM, planilhas, bot, assinatura eletrônica e Oracle no padrão; fct_contatos, dim_clientes com LGPD, mart de efetividade; policy tags e authorized views por credor; Slim CI; alertas de freshness e de teste.	Todas as carteiras na plataforma; nenhum dashboard lendo CPF em claro; onboarding de fonte nova em até 1 semana.
3. Operação e IA (sem. 9-13)	Fila de cobrança do dia servida da Curated; features de propensão e melhor canal/horário no Feature Store; piloto de embeddings sobre transcrições com DLP; painel de FinOps; decisão sobre Composer; documentação e passagem de bastão.	Operação usando a fila; propensão em teste A/B em uma carteira; custo mensal conhecido e dentro do orçamento.

5. Riscos e mitigações

Risco	Mitigação
Fornecedor muda layout sem aviso	Landing imutável preserva o que chegou; testes de staging falham antes do mart; catálogo de resultados em tabela de referência isola o MIS; contrato de dados por fornecedor.
Oracle sem janela ou sem auditoria confiável	Negociar janela e sessões com o DBA na semana 1; fatias com retomada; hash por linha; medir volume antes de prometer latência.
Indicadores não batem com o MIS atual	Operação paralela com reconciliação automatizada por indicador e divergências explicadas por escrito antes de desligar o antigo.
Exposição de dado pessoal	Curated sem dado em claro por construção e por teste; policy tags; auditoria de leitura; revisão LGPD em todo PR que toque dim_clientes.
Custo cresce com consultas do BI	Partição e cluster; maximum_bytes_billed; cota por usuário; painel de custo por consulta na primeira semana.
Dependência de uma pessoa / IA decidindo sozinha	Tudo em Git com docs e CI; passagem de bastão no bloco 3. Modelo prioriza fila, humano decide; monitoramento de viés por carteira.

6. O que já está construído no repositório

- Projeto dbt sobre BigQuery com dataset sintético de cobrança (600 clientes, 900 contratos, ~11 mil parcelas, 149 acordos, ~8,2 mil pagamentos, ~17 mil contatos de discador, URA, bot, SMS e e-mail).
- 16 modelos em Raw/Cleansed/Curated; régua e catálogo de resultados como tabelas de referência; macros para sandbox, data de referência, hash LGPD e faixas de aging.
- 153 testes + teste genérico próprio + 3 singulares: reconciliação financeira entre camadas, ausência de dado pessoal em claro (via INFORMATION_SCHEMA) e integridade de acordos quitados.
- **dbt build executado no BigQuery: PASS=177, WARN=0, ERROR=0** (8 seeds, 16 modelos, 153 testes). O teste de reconciliação apontou, durante a construção, dois erros que deixariam o número "bonito e errado".
- Ingestão de planilha (CSV/Excel) e de API REST incremental (paginação, retry, marca d'água) para landing local ou gs://, com mock de API e execução no CI do GitHub Actions (dbt parse + ingestão, sem credenciais).

Proposta, não relato de projeto entregue: minha experiência em cobrança vem de atuar como a pessoa de dados do canal digital de cobrança (discador, bot, fontes e indicadores) e de 15+ anos em MIS/BI, SQL Server/PostgreSQL, Python e Medallion na área financeira; Composer, Vertex AI e Oracle são escolhas de arquitetura estudadas para este desenho.

Sidney Moraes · linkedin.com/in/sid-moraes · github.com/sysdevcns/plataforma-dados-cobranca-gcp